



자바응용SW(앱)개발자양성

AdapterView 활용

백제직업전문학교

김영준 강사



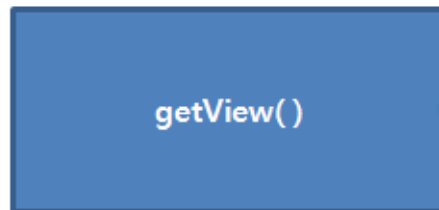
왜 굳이 선택위젯이라는 이름으로 구분할까?

- 안드로이드에서는 여러 아이템 중의 하나를 선택하는 선택위젯은 별도의 패턴을 사용함
- 여러 개의 아이템 중에서 하나를 선택하는 방식의 선택 위젯은 어댑터를 사용하여야 함
- 이 어댑터에서 데이터를 관리하도록 해야 할 뿐만 아니라 화면에 보여지는 뷰도 어댑터의 `getView()` 메소드에서 결정함
- 선택위젯의 가장 큰 특징은 원본 데이터를 위젯에 직접 설정하지 않고 어댑터라는 클래스를 사용하도록 되어 있다는 점으로 이 패턴을 잘 기억해 두어야 함

원본 데이터



데이터 어댑터



선택 위젯



[선택 위젯과 어댑터]



리스트뷰로 보여줄 때 해야 할 일들

(1) 아이템을 위한 XML 레이아웃 정의하기

- 리스트뷰에 들어갈 각 아이템의 레이아웃을 XML로 정의함

(2) 아이템을 위한 뷰 정의하기

- 리스트뷰에 들어갈 각 아이템을 하나의 뷰로 정의. 이 뷰는 여러 개의 뷰를 담고 있는 뷰그룹이어야 함
- 이것은 부분화면과 같아서 (1)번에서 정의한 XML 레이아웃을 인플레이션 후 설정해야 함

(3) 어댑터 정의하기

- 데이터 관리 역할을 하는 어댑터 클래스를 만들고 그 안에 각 아이템으로 표시할 뷰를 리턴하는 getView() 메소드를 정의함

(4) 리스트뷰 정의하기

- 화면에 보여줄 리스트뷰를 만들고 그 안에 데이터가 선택되었을 때 호출될 리스너 객체를 정의함



리스트뷰 사용하기 예제

리스트뷰 사용하기 예제

- 아이콘이 들어 있는 리스트뷰의 아이템 정의
- 리스트뷰의 한 아이템을 선택했을 때 토스트 표시

아이템의 XML 레이아웃

- 한 아이템으로 보여질 뷰의 XML 레이아웃 정의

한 아이템의 데이터를 넣을 클래스

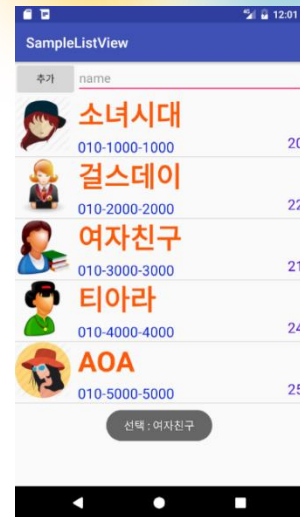
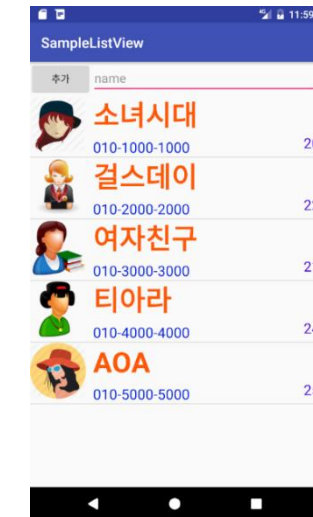
- 한 아이템의 데이터를 넣을 클래스 정의

어댑터 클래스 정의

- 리스트뷰를 보여주는 역할을 하는 어댑터 클래스 정의

리스트뷰를 사용하는 메인 액티비티 코드 작성

- 리스트뷰를 사용하는 메인 액티비티 코드 작성



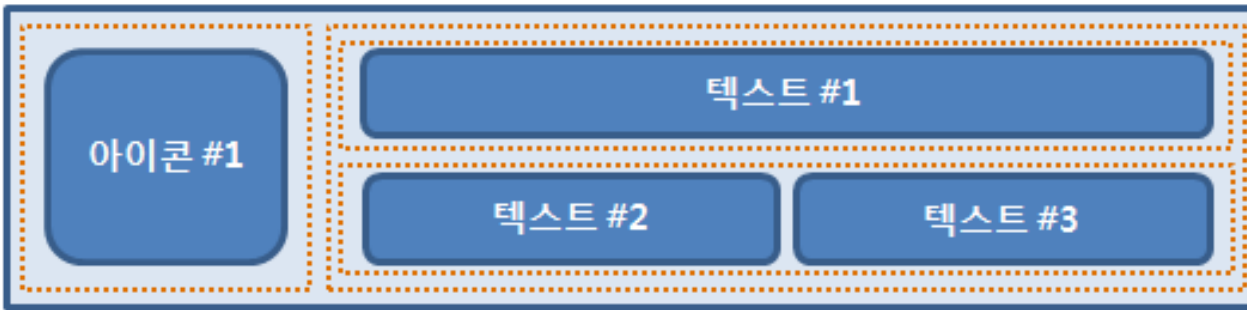
리스트뷰 선택 이벤트 처리 코드 작성

- 한 아이템을 선택했을 때의 이벤트 처리



한 아이템으로 보여줄 XML 레이아웃 정의

- 선택위젯에서 각각의 아이템은 동일한 레이아웃을 가진 뷰가 반복적으로 보여짐
- 각각의 아이템을 위한 XML 레이아웃이 필요함



[리스트뷰의 한 아이템이 갖는 레이아웃의 예]

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    >
```

Continued..



한 아이템으로 보여줄 XML 레이아웃 정의 (계속)

```
<ImageView
```

```
    android:id="@+id/iconItem"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:padding="8dip"
```

```
    android:layout_gravity="center_vertical"
```

```
/>
```

1 왼쪽에 보이는 이미지뷰 정의

```
<LinearLayout
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:orientation="vertical"
```

```
    android:layout_alignParentLeft="true"
```

```
>
```

Continued..



한 아이템으로 보여줄 뷰 정의

- 어댑터의 getView() 메소드에서 리턴해 줄 뷰를 별도의 클래스로 정의하여 사용

```
public class SingerItemView extends LinearLayout {  
    ...  
    LayoutInflater inflater = (LayoutInflater)  
        context.getSystemService(Context.LAYOUT_INFLATER_SERVICE);  
    inflater.inflate(R.layout.singer_item, this, true);  
  
    imageView = (ImageView) findViewById(R.id.imageView);  
    ...  
}
```



한 아이템으로 보여줄 데이터 클래스 정의

- 각각의 아이템을 위한 데이터도 별도의 클래스로 정의하여 사용

```
public class SingerItem {  
  
    String name;  
    String mobile;  
    int age;  
    int resId;  
  
    public SingerItem(String name, String mobile, int age, int resId) {  
        this.name = name;  
        this.mobile = mobile;  
        this.age = age;  
        this.resId = resId;  
    }  
}
```




새로운 어댑터 클래스 정의

```
public class SingerAdapter extends BaseAdapter {  
    private Context context;  
  
    private ArrayList<SingerItem> items = new ArrayList<SingerItem>();  
  
    public SingerAdapter(Context context) {  
        context = context;  
    }  
    ...  
    public int getCount() {  
        return items.size();  
    }  
    ...  
    public Object getItem(int position) {  
        return items.get(position);  
    }  
    ...  
}
```

Continued..



새로운 어댑터 클래스 정의 (계속)

@Override

```
public View getView(int position, View convertView, ViewGroup viewGroup) {  
    SingerItemView view = new SingerItemView(getApplicationContext());  
  
    SingerItem item = items.get(position);  
    view.setName(item.getName());  
    view.setMobile(item.getMobile());  
    view.setAge(item.getAge());  
    view.setImage(item.getResId());  
  
    return view;  
}
```



getView() 메소드

[Reference]

```
public View getView (int position, View convertView, ViewGroup parent)
```

- 첫 번째 파라미터 - 아이템의 인덱스를 의미하는 것으로 리스트뷰에서 보일 아이템의 위치 정보라 할 수 있음.
0부터 시작하여 아이템의 개수만큼 파라미터로 전달됨
- 두 번째 파라미터 - 현재 인덱스에 해당하는 뷰 객체를 의미하는데 안드로이드에서는 선택 위젯이 데이터가 많아 스크롤될 때 뷰를 재활용하는 메커니즘을 가지고 있어 한 번 만들어진 뷰가 화면 상에 그대로 다시 보일 수 있도록 되어 있음. 따라서 이 뷰가 널값이 아니면 재활용 가능함
- 세 번째 파라미터 - 부모 컨테이너 객체임



메인 액티비티 코드 만들기

```
listView = (ListView) findViewById(R.id.listView);
```

```
adapter = new SingerAdapter();
```

```
adapter.addItem(new SingerItem("소녀시대", "010-1000-1000", 20, R.drawable.singer));
```

```
adapter.addItem(new SingerItem("걸스데이", "010-2000-2000", 22, R.drawable.singer2));
```

```
adapter.addItem(new SingerItem("여자친구", "010-3000-3000", 21, R.drawable.singer3));
```

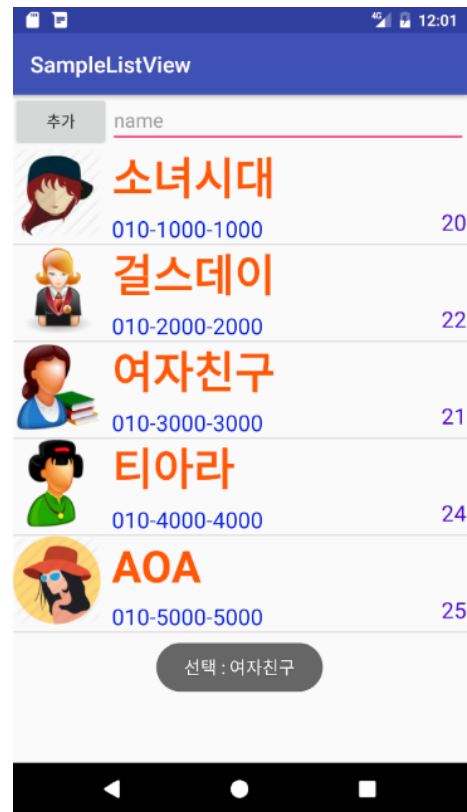
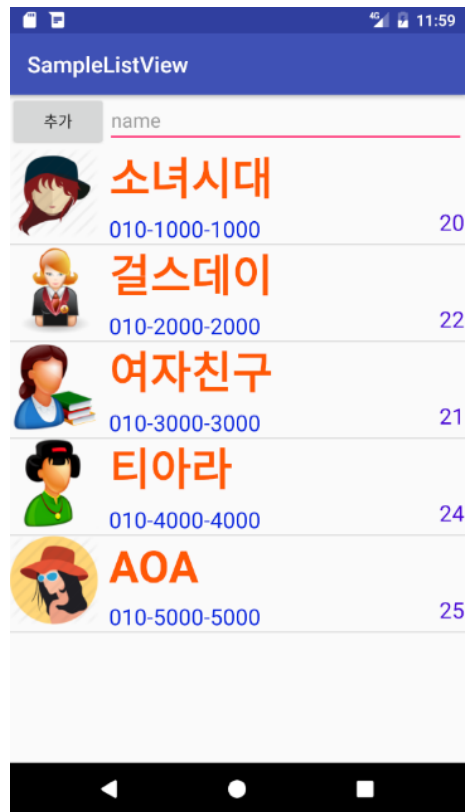
```
adapter.addItem(new SingerItem("티아라", "010-4000-4000", 24, R.drawable.singer4));
```

```
adapter.addItem(new SingerItem("AOA", "010-5000-5000", 25, R.drawable.singer5));
```

```
listView.setAdapter(adapter);
```



실행 화면





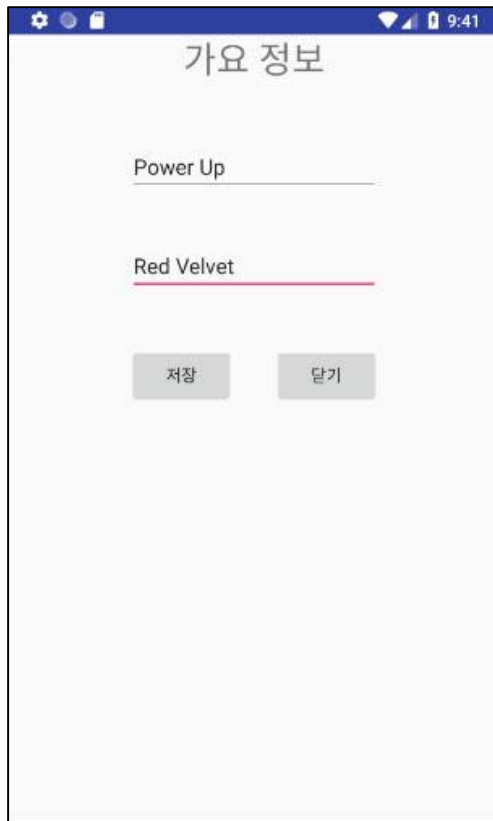
리스트뷰로 화면만들기

1. activity_main.xml 파일을 열고 리스트뷰를 이용해서 가요 리스트를 보여줄 수 있도록 구성합니다.
2. 가요 정보가 표시되는 아이템은 왼쪽에 사진이 있고 오른쪽에 제목, 가수명이 표시되도록 합니다.
3. 가장 아래쪽에는 버튼 1개를 더 추가합니다. 버튼에는 추가라는 텍스트를 표시합니다.
추가 버튼을 누르면 가요 정보를 추가할 수 있는 새로운 화면이 뜹니다
4. MainActivity.java 파일을 열고 리스트뷰를 구현하기 위한 어댑터 클래스를 만듭니다
5. 새로운 화면에는 가요 이름과 가수 이름을 입력할 수 있도록 하고 저장 버튼을 누르면 메인 화면으로 돌아오면서 리스트뷰에 추가되도록 합니다
6. 새로운 화면에서 받은 정보를 이용해 리스트뷰에 아이템을 추가하는 코드를 onActivityResult 메서드 안에 입력합니다
7. 리스트뷰의 한 아이템을 선택했을 때 토스트 메시지를 보여주도록 하기 위해 리스트뷰 객체에 onItemClickListener 객체를 설정합니다



리스트뷰로 화면만들기

<출력화면>





자바응용SW(앱)개발자양성

RecyclerView

백제직업전문학교

김영준 강사

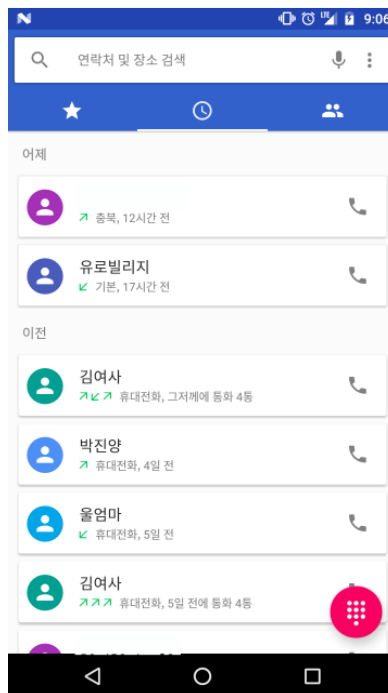


RecyclerView

1. RecyclerView 소개

- RecyclerView는 API Level 21(Android 5.0)이 나오면서 androidx.recyclerview 라이브러리로 제공된 클래스로

implementation 'androidx.recyclerview:recyclerview:1.1.0'





RecyclerView



- Adapter: RecyclerView 항목 구성
- ViewHolder: 각 항목 구성 뷰의 재활용을 목적으로 View Holder 역할
- LayoutManager : 항목의 배치
- ItemDecoration: 항목 꾸미기
- ItemAnimation: 아이템이 추가, 제거, 정렬될 때의 애니메이션 처리

2. Adapter, ViewHolder

- RecyclerView를 하나 준비

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recyclerView"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" />
```



RecyclerView

- ViewHolder 클래스

```
private class MyViewHolder extends RecyclerView.ViewHolder {  
    public TextView title;  
    public MyViewHolder(View itemView) {  
        super(itemView);  
        title = (TextView) itemView.findViewById(android.R.id.text1);  
    }  
}
```

- Adapter

```
private class MyAdapter extends RecyclerView.Adapter<MyViewHolder> {  
    private List<String> list;  
    public MyAdapter(List<String> list) {  
        this.list = list;  
    }  
    public MyViewHolder onCreateViewHolder(ViewGroup viewGroup, int i) {  
        View view = LayoutInflater.from(viewGroup.getContext())  
            .inflate(android.R.layout.simple_list_item_1, viewGroup, false);  
        return new MyViewHolder(view);  
    }  
    public void onBindViewHolder(MyViewHolder viewHolder, int position) {  
        String text = list.get(position);  
        viewHolder.title.setText(text);  
    }  
    public int getItemCount() {  
        return list.size();  
    }  
}
```



RecyclerView

```
private class MyAdapter extends RecyclerView.Adapter<MyViewHolder> {  
    //.....  
    layout inflate ① public MyViewHolder onCreateViewHolder(ViewGroup viewGroup, int i) {  
        View view = LayoutInflater.from(viewGroup.getContext())  
        ViewHolder 리턴 ④ .inflate(android.R.layout.simple_list_item_1, viewGroup, false);  
        return new MyViewHolder(view);  
    }  
    @Override  
    항목 구성 ⑤ public void onBindViewHolder(MyViewHolder viewHolder, int position) {  
        String text = list.get(position);  
        viewHolder.title.setText(text);  
    } ⑥ 전달된 ViewHolder 이용 ② Layout 초기화후 ViewHolder 생성  
}  
private class MyViewHolder extends RecyclerView.ViewHolder {  
    public TextView title;  
    public MyViewHolder(View itemView) {  
        super(itemView);  
        title = (TextView) itemView.findViewById(android.R.id.text1);  
    }  
}
```

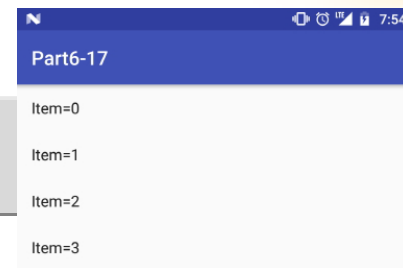
③ 필요 View findViewById



RecyclerView

- Adapter를 RecyclerView에 적용

```
recyclerView.setLayoutManager(new LinearLayoutManager(this));  
recyclerView.setAdapter(new MyAdapter(list));
```



3. LayoutManager

- 항목을 어떻게 배치
- LinearLayoutManager: 수평, 수직으로 배치
- GridLayoutManager: 그리드 화면으로 배치
- StaggeredGridLayoutManager: 높이가 불규칙한 그리드 화면으로 배치
- LinearLayoutManager

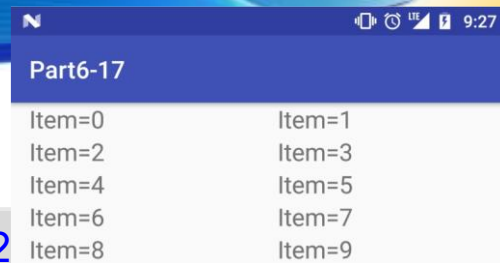
```
LinearLayoutManager linearManager = new LinearLayoutManager(this);  
linearManager.setOrientation(LinearLayoutManager.HORIZONTAL);  
recyclerView.setLayoutManager(linearManager);
```



RecyclerView

- GridLayoutManager

```
GridLayoutManager gridManager=new GridLayoutManager(this, 2,
recyclerView.setLayoutManager(gridManager);
```



Part6-17	
Item=0	Item=1
Item=2	Item=3
Item=4	Item=5
Item=6	Item=7
Item=8	Item=9

- 방향 지정

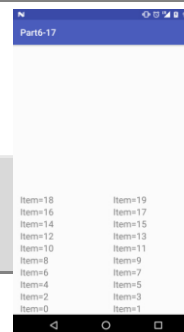
```
GridLayoutManager gridManager=new GridLayoutManager(this, 2, GridLayoutManager.HORIZONTAL, false);
```



Part6-17				
Item=0	Item=2	Item=4	Item=6	Item=8
Item=1	Item=3	Item=5	Item=7	Item=9

- 아래부터 나열

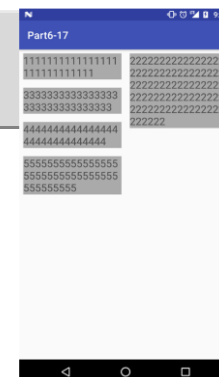
```
GridLayoutManager gridManager=new GridLayoutManager(this, 2,
GridLayoutManager.VERTICAL, true);
```



Part6-17	
Item=18	Item=19
Item=16	Item=17
Item=14	Item=15
Item=12	Item=13
Item=10	Item=11
Item=8	Item=9
Item=6	Item=7
Item=4	Item=5
Item=2	Item=3
Item=0	Item=1

- StaggeredGridLayoutManager

```
StaggeredGridLayoutManager sgManager=new StaggeredGridLayoutManager(2,
StaggeredGridLayoutManager.VERTICAL);
```



Part6-17	
11111111111111111111	22222222222222222222
11111111111111111111	22222222222222222222
33333333333333333333	22222222222222222222
33333333333333333333	22222222222222222222
44444444444444444444	22222222222222222222
44444444444444444444	222222
55555555555555555555	
55555555555555555555	
55555555	



RecyclerView

4. ItemDecoration

각 항목을 다양하게 꾸미기

- onDraw: 항목을 배치하기 전에 호출
- onDrawOver: 모든 항목이 배치된 후에 호출
- getItemOffsets: 각 항목을 배치할 때 호출

```
class MyItemDecoration extends RecyclerView.ItemDecoration {  
  
    @Override  
    public void getItemOffsets(Rect outRect, View view, RecyclerView parent,  
                               RecyclerView.State state) {  
        super.getItemOffsets(outRect, view, parent, state);  
        //항목의 index 값 획득  
        int index=parent.getChildAdapterPosition(view)+1;  
  
        if(index % 3 == 0)  
            //left, top, right, bottom  
            outRect.set(20, 20, 20, 60 );  
        else  
            outRect.set(20, 20, 20, 20 );  
  
        view.setBackgroundColor(0xFFECE9E9);  
        ViewCompat.setElevation(view, 20.0f);  
    }  
}
```




RecyclerView

- ItemDecoration을 RecyclerView에 적용

```
recyclerView.addItemDecoration(new MyItemDecoration());
```



- onDraw() 함수

```
public void onDraw(Canvas c, RecyclerView parent, RecyclerView.State state) {  
    super.onDraw(c, parent, state);  
    //RecyclerView의 사이즈 계산  
    int width=parent.getWidth();  
    int height=parent.getHeight();  
  
    Paint paint=new Paint();  
    paint.setColor(Color.RED);  
    c.drawRect(0, 0, width/3, height, paint );  
    paint.setColor(Color.BLUE);  
    c.drawRect(width/3, 0, width/3*2, height, paint);  
    paint.setColor(Color.GREEN);  
    c.drawRect(width/3*2, 0, width, height, paint);  
}
```

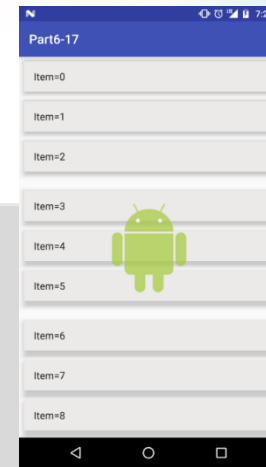




RecyclerView

- onDrawOver()

```
public void onDrawOver(Canvas c, RecyclerView parent, RecyclerView.State state) {  
    super.onDrawOver(c, parent, state);  
    //RecyclerView의 사이즈 계산  
    int width=parent.getWidth();  
    int height=parent.getHeight();  
    //이미지 사이즈 계산  
    Drawable dr= ResourcesCompat.getDrawable(getResources(), R.drawable.android, null);  
    int drWidth=dr.getIntrinsicWidth();  
    int drHeight=dr.getIntrinsicHeight();  
  
    int left=width/2 - drWidth/2;  
    int top=height/2 - drHeight/2;  
    c.drawBitmap(BitmapFactory.decodeResource(getResources(), R.drawable.android), left,top,null);  
}
```





RecyclerView

RecyclerView 테스트

